

Arquitetura do Sistema de Apoio à Pesquisa Jurídica (TJSP)

1. Módulo de Coleta e Ingestão (Scraping)

O objetivo desta camada é espelhar os autos públicos do e-SAJ de forma ética, legal e resiliente.

- **Entrada:** Número do Processo (Padrão CNJ: NNNNNNN-DD.AAAA.J.TR.OOOO).
- **Mecanismo de Navegação:** Automação headless controlada via filas de mensageria para acessar as consultas públicas de 1º e 2º Grau do TJSP.
- **Regras de Cidadania Digital:**
 - Não há burla de CAPTCHA ou login automatizado em áreas restritas. Apenas processos e documentos públicos são lidos.
 - Uso de Rate Limiting (intervalos entre requisições) e Exponential Backoff para lidar com indisponibilidades (HTTP 503/429) sem onerar os servidores do tribunal.
- **Extração de Metadados:** Captura de classe, assunto, foro, vara, juiz, partes e histórico de movimentações da página HTML.
- **Download de Documentos:** Identificação da árvore da "Pasta Digital". Download dos PDFs públicos diretamente para um Object Storage (armazenamento em nuvem no formato bruto).
- **Atualização (Delta):** Em consultas futuras, o sistema verifica a data da última movimentação. Se houver diferença, baixa apenas os PDFs novos.

2. Armazenamento e Modelagem de Dados (Camada de Persistência)

A separação entre dados estruturados (relacionais), binários e vetoriais é vital para o RAG Híbrido.

- **Object Storage (ex: S3/MinIO):** Armazena os PDFs originais (Brutos) e PDFs processados (limpos).
- **Banco de Dados Relacional (PostgreSQL):**
 - processos: id, cnj, classe, assunto, foro, juiz, valor_causa, data_distribuicao.
 - partes: id, processo_id, tipo (autor/reu), nome.
 - movimentacoes: id, processo_id, data, descricao.
 - documentos: id, processo_id, tipo_classificado (Petição, Sentença, etc.), data_juntada, url_s3, hash_arquivo.

- **Banco Vetorial (pgvector dentro do PostgreSQL):**

- chunks: id, documento_id, texto_trecho, vetor_embedding, numero_pagina, agente_emissor, secao_documento.

3. Pipeline de Processamento Documental (Orquestração com LangGraph)

Esta é a esteira que transforma o PDF desestruturado em conhecimento estruturado. O LangGraph orquestra este fluxo garantindo que cada documento siga regras específicas.

1. **Extração de Texto (Parsing/OCR):** Tenta ler o texto nativo (PyMuPDF). Se for imagem (petição escaneada), aciona um OCR de alta precisão para português (DocTR ou Tesseract).
2. **Limpeza (Sanitização):** Remove numeração de margem do TJSP, assinaturas digitais, cabeçalhos e rodapés que quebram frases no meio.
3. **Classificação do Documento (SLM / Heurística):** Analisa a folha de rosto para definir se é "Petição Inicial", "Contestação", "Sentença", etc.
4. **Role Tagging (Atribuição de Autoria):** Associa a Inicial ao Autor, a Contestação ao Réu, a Sentença ao Magistrado.
5. **Chunking Jurídico e Estrutural:** O texto não é cortado a cada 1000 caracteres. Ele é segmentado por estrutura jurídica (ex: quebra em "Dos Fatos", "Do Direito"). Sentenças são divididas em "Relatório" e "Fundamentação/Dispositivo".
6. **Vetorização:** Converte os chunks textuais em vetores matemáticos usando um modelo de embeddings.

4. RAG Jurídico Híbrido (O Motor de Busca)

Quando o advogado faz uma pergunta (ex: "Quais os fundamentos para negar a prescrição?"):

1. **Tradução da Pergunta (Query Understanding):** A IA entende que o alvo é "Fundamento" -> aplica filtro rígido: buscar apenas em documentos com agente_emissor = Magistrado.
2. **Busca Lexical (BM25):** Procura correspondências exatas de palavras-chave (ex: "prescrição", "Art. 206").
3. **Busca Semântica (Embeddings):** Procura pelo significado (ex: "perda do prazo", "decadência").
4. **Reranking (Cross-Encoder):** Combina os resultados léxicos e semânticos (Top 50) e usa um modelo especialista para reordená-los comparando intimamente a pergunta com cada trecho, extraíndo o Top 10 mais relevante.

5. Geração de Resposta e Rastreabilidade (LLM)

O Top 10 de trechos (Chunks) é enviado para o LLM.

- **Prompt com Amnésia Induzida:** A LLM é instruída por sistema a não usar conhecimento prévio do mundo. Ela só pode responder baseada no XML injetado.
- **Citações Obrigatórias:** A LLM deve ancorar cada resposta no formato [Documento, p. X].
- **Frontend (Interface):** O frontend converte a citação em um botão. Ao clicar, a tela divide e exibe o PDF original com o parágrafo exato destacado em amarelo.

6. Fluxo de Execução de Ponta a Ponta

1. **Input:** O usuário insere o CNJ 1012345-67.2024... no sistema.
2. **Coleta:** O scraper entra no e-SAJ, baixa os dados e PDFs públicos.
3. **ETL:** O LangGraph limpa, classifica, recorta em chunks semânticos e salva no PostgreSQL/pgvector.
4. **Consulta:** Usuário digita: "Quais os argumentos da ré?"
5. **Filtro:** Backend gera a query `SELECT ... WHERE emissor = 'Réu' AND id_processo = X`.
6. **Busca:** O pgvector + BM25 retornam os trechos da contestação. O Reranker ajusta a ordem.
7. **Geração:** O LLM lê apenas a contestação e escreve a síntese das teses de defesa com as citações.
8. **Output:** A tela mostra a resposta estruturada. O advogado clica na citação e valida no PDF.

7. Diagrama Textual da Arquitetura

```
[ INTERFACE DO USUÁRIO (React/Vue) ]
|
| (1) Pedido de Busca/Chat (REST/GraphQL)
v
[ API BACKEND (FastAPI - Python) ] <===== [ SERVIÇOS DE AUDITORIA & LOGS ]
|
| (2) Dispara Scraping Assíncrono
|
| [ WORKER CELERY ] ---> [ SCRAPER HEADLESS ] ---> ( TRIBUNAL DE JUSTIÇA / e-SAJ )
|
| +-- (Grava PDFs) --> [ OBJECT STORAGE (S3/MinIO) ]
```

```

|
|-- (3) Inicia Processamento
|   v
|   [ PIPELINE DE DADOS (LangGraph) ]
|     |-- Extrator Textual / OCR (PyMuPDF / DocTR)
|     |-- Classificador de Peças (SLM)
|     |-- Role Tagger (Atribui Autor/Réu/Juiz)
|     |-- Chunking Semântico
|     +-- Embedding Generator (BGE-M3)
|
|-- (4) Consulta RAG Híbrida
|   v
|   [ MOTOR DE BUSCA ]
|     |-- [ POSTGRESQL ] (Metadados Estruturados + Partes + Movimentações)
|     |-- [ PGVECTOR ] (Busca Semântica)
|     +-- [ BM25 ] (Busca Lexical)
|   v
|   [ RERANKER (Cross-Encoder) ]
|
+-- (5) Geração de Resposta
    v
    [ LLM ENGINE (vLLM / Ollama) ]
      |-- System Prompt Anti-Alucinação
      +-- Retorna Resposta com Citações [Doc, p. X]

```

8. Stack Tecnológico Recomendado (Open Source e Soberano)

- **Backend & APIs:** Python 3.11+ com FastAPI (alta performance, tipagem forte, assíncrono).
- **Workers & Tarefas em 2º Plano:** Celery com mensageria Redis (para gerenciar as filas de scraping e processamento).
- **Banco de Dados Principal:** PostgreSQL 16 com a extensão pgvector. Combina relacionamentos complexos com busca vetorial no mesmo local.
- **Orquestração de LLM:** LangChain e LangGraph (para o fluxo condicional de documentos).
- **Extração e OCR:** PyMuPDF (rápido para PDFs nativos) e DocTR (robusto para PDFs escaneados).
- **Modelos de Embedding e Reranking:** BGE-M3 (multilíngue) e bge-reranker-v2-m3.
- **Motor LLM:** Ollama ou vLLM com o modelo Llama-3-70B-Instruct rodando localmente para total privacidade (ou APIs externas controladas).

9. Métricas de Avaliação e Testes

- **Faithfulness (Fidelidade):** O que a IA afirmou está contido nos chunks apresentados? (Mede a alucinação).
- **Answer Relevance:** A resposta atende à pergunta do usuário ou fugiu do tema?
- **Context Precision:** O banco de dados encontrou a página certa da sentença para enviar à IA?
- **Taxa de Sucesso do Role Tagging:** O sistema confundiu alguma vez a alegação do advogado com a decisão do juiz? (Deve ser 0%).

10. Estratégia de Entregas (MVP vs Fase 2)

Fase 1: MVP (Mínimo Produto Viável)

Foco: Fazer um processo funcionar de ponta a ponta perfeitamente.

- Scraper básico capaz de entrar no TJSP com 1 CNJ e baixar a pasta digital pública.
- Banco de Dados configurado com pgvector.
- Pipeline capaz de processar PDFs nativos e PDFs escaneados legíveis.
- Classificação apenas das 3 peças fundamentais: Petição Inicial, Contestação e Sentença.
- Interface de chat onde o advogado faz perguntas sobre aquele processo específico e recebe respostas com citações e metadados.

Fase 2: Expansão e Inteligência de Padrões

Foco: Cruzamento de dados e análise avançada.

- Expansão do Scraper para buscar Acórdãos (2ª Instância).
- Módulo de Processos Semelhantes (Cross-Process RAG): Identificar automaticamente onde uma tese da defesa funcionou contra o mesmo juiz em processos anteriores.
- Detecção automatizada de teses repetitivas.
- Painel de Analytics (ex: tempo médio de tramitação comparado entre varas).